

MediQueueAI: AI-Powered Healthcare and Patient Information Management System

A Project Based Learning Report Submitted in partial fulfilment of the requirements for the award of the degree

of

Bachelor of Technology

in The Department of Electronics and Communication Engineering

DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT - 25CS1302E

Submitted by

2500030017: Suhaanthi Reddy

2510030107: Spoorthy Reddy

2510020273: Prasanna Reddy

2510030302: Sreeja

2510030312: DhanaLakshmi

Under the guidance of

Dr. P Lalitha Surya Kumari



Department of Electronics and Communication Engineering

Koneru Lakshmaiah Education Foundation, Aziz Nagar

Aziz Nagar – 500075

OCT - 2026.

Abstract

MediQueueAI is an AI-assisted healthcare and patient information management prototype that connects patient records with clinical encounters, risk assessment, emergency prioritization, and treatment tracking. In its database-driven workflow, patient information, symptoms, vital signs, assessment outputs, prescriptions, treatment status, and reports remain associated with the selected patient encounter.

The system uses a React, TypeScript, and Vite frontend; a FastAPI backend exposing REST APIs; and a PostgreSQL database. The assessment workflow takes heart rate, blood pressure, oxygen saturation (SpO₂), respiratory rate, temperature, and symptoms, and produces a risk score, risk level, priority score, priority level, and an explanation. The priority categories are Critical, High, Moderate, and Stable. A priority queue orders active cases by urgency and is updated when treatment is completed.

The methodology follows four stages: retrieval of clinical data for the selected encounter, AI-assisted assessment, classification into a priority category, and a database update that stores the assessment, reorders the queue, and tracks treatment status. The prototype covers patient management, encounter and clinical data, assessment, priority queue, doctor assignment, treatment tracking, reports, and system-status monitoring, with each result kept traceable to the patient encounter that produced it.

A demonstration case with SpO₂ of 89%, heart rate of 128 beats per minute, blood pressure of 88/60, and respiratory rate of 26 per minute is assigned a risk score of 100 and Critical priority. This is a demonstration example, not clinical validation. The project aims to improve information organization, help staff identify urgent cases, reduce repetitive record searching, and support a more connected care workflow. Further testing, clinical review, security assessment, and validation with representative data would be required before real-world clinical use.

Keywords: *healthcare information management, AI-assisted triage, PostgreSQL, patient encounters, risk scoring, priority queue, FastAPI.*

List of Figures

Fig. 1. End-to-end MediQueueAI workflow.	7
Fig. 2. Four-stage methodology flowchart.	8
Fig. 3. Data retrieval and verification flowchart.	9
Fig. 4. Clinical data to risk and priority classification flowchart.	10
Fig. 5. Priority queue update flowchart.	11
Fig. 6. Layered system architecture.	11
Fig. 7. Patient–encounter information flow.	13
Fig. 8. Illustrative MediQueueAI dashboard interface (sample data).	14
Fig. 9. Illustrative AI risk and priority assessment screen (sample data).	15
Fig. 10. Demonstration case risk–priority summary.	15
Fig. 11. Illustrative emergency priority queue interface (sample data).	16

List of Tables

TABLE I. Project objectives and expected contributions	6
TABLE II. Functional requirements.....	8
TABLE III. Non-functional requirements	8
TABLE IV. Clinical input data.....	9
TABLE V. Risk and priority levels	10
TABLE VI. Technology stack.....	12
TABLE VII. Core data entities and relationships.....	12
TABLE VIII. Core modules of the prototype.....	14
TABLE IX. Demonstration case values	15
TABLE X. Evaluation summary	17
TABLE XI. Limitations and future enhancements.....	18

Table of Contents

1. INTRODUCTION	6
1.a. Background and Motivation	6
1.b. Problem Statement.....	6
1.c. Objectives	6
1.d. Scope of the Project.....	6
1.e. Organization of the Report	7
2. METHODOLOGY	7
2.a. Existing Workflow and Challenges	7
2.b. Proposed System.....	7
2.c. Functional Requirements	8
2.d. Non-Functional Requirements.....	8
2.e. Four-Stage Methodology	8
2.f. Stage 1: Clinical Data	9
2.g. Data Retrieval and Verification	9
2.h. Stage 2: AI Assessment	9
2.i. Stage 3: Priority Classification	10
2.j. Stage 4: Database Update and Priority Queue.....	10
2.k. System Architecture	11
2.l. Technology Stack	12
2.m. Data Model and Information Flow	12
3. EXPERIMENTS.....	14
3.a. Core Modules	14
3.b. End-to-End Interaction	14
3.c. Demonstration Case.....	15
3.d. Demonstration Walkthrough	16
3.e. Interpretation	16
4. RESULTS.....	16
4.a. Observed Prototype Capabilities	16
4.b. Benefits.....	16
4.c. Evaluation Summary	17
4.d. Discussion.....	17
5. CONCLUSION AND FUTURE WORK.....	17
5.a. Limitations, Clinical Safety and Privacy	17
5.b. Future Work.....	18
REFERENCES.....	19

MediQueueAI: AI-Powered Healthcare and Patient Information Management System

1. INTRODUCTION

1.a. Background and Motivation

Healthcare facilities handle patient registration, clinical observations, appointments, assessments, prescriptions, and treatment progress. When these details are stored or viewed separately, staff may need to repeat searches and manually determine which cases require attention first. In busy settings such as emergency care, this makes it harder to keep urgency visible when several patients are waiting, and it increases the effort needed to find the right record at the right time.

The MediQueueAI project addresses this workflow challenge by presenting patient and encounter data together with AI-assisted risk and priority assessment [1], [2]. It is designed as a database-driven patient management application rather than an isolated prediction screen. A result is intended to remain connected to the patient and the encounter from which the clinical inputs were retrieved. This traceability is important for reviewing the information used by the assessment and for following the case through treatment and reporting [1].

1.b. Problem Statement

The project seeks to reduce fragmentation in patient information management and support faster recognition of high-risk cases. A conventional manual workflow can require staff to search records, interpret symptoms and vital signs, and separately communicate urgency. MediQueueAI proposes a connected workflow in which structured clinical data is assessed, mapped to understandable risk and priority levels, and used to organize the patient queue [1], [2].

1.a. Objectives

The objectives of the project and the contribution expected from each are summarized in Table I.

TABLE I. PROJECT OBJECTIVES AND EXPECTED CONTRIBUTIONS

Objective	Expected contribution
Centralize patient information	Keep patient, encounter, clinical, assessment, and treatment details connected.
Support risk assessment	Use symptoms and vital signs as inputs to an AI-assisted assessment.
Prioritize urgent cases	Represent risk and priority through Critical, High, Moderate, and Stable categories.
Support clinical workflow	Connect assessment outcomes with doctor assignment, prescriptions, treatment status, and reports.
Improve traceability	Retrieve data for the selected patient encounter and associate the result with that encounter.
Support extension	Use a modular frontend/backend structure and database-backed records to support future growth.

1.b. Scope of the Project

The scope of this report covers patient information retrieval, encounter-level clinical data, AI-assisted assessment on request, risk and priority scores, priority queue ordering, treatment-status updates, and reporting. The system also monitors API health and PostgreSQL connectivity. The implementation uses React, TypeScript, and Vite for the frontend, FastAPI for the backend REST APIs, and PostgreSQL for persistent storage [1], [2].

This report documents the design, workflow, and demonstration of the prototype. It does not report model training data, model family, validation metrics, clinical approval, authentication controls, encryption configuration, or deployment environment, and none of these should be inferred from it. MediQueueAI is therefore evaluated as a software prototype and decision-support aid, not as an autonomous diagnostic system.

1.c. Organization of the Report

The Methodology section describes the existing workflow, the proposed system, the requirements, the four-stage assessment methodology, the architecture, and the data model. The Experiments section presents the core modules and the demonstration case. The Results section summarizes the observed prototype capabilities and evaluates the available evidence. The Conclusion and Future Work section states the limitations, clinical-safety and privacy considerations, and recommended enhancements.

1. METHODOLOGY

1.a. Existing Workflow and Challenges

In a fragmented workflow, patient registration, symptoms, vital signs, prescriptions, and treatment updates may be recorded in separate places or viewed through different screens. Staff may spend time locating the correct encounter and determining whether a case is routine or urgent. Manual queue ordering can also make it harder to keep urgency visible when multiple patients are waiting.

The project contrasts this pattern with a centralized patient record and priority-based care. The proposed system keeps patient information, symptoms, vitals, assessment, prescriptions, treatment status, and reports within a connected workflow. The key design principle is that an assessment should be traceable to the encounter used to generate it [2].

1.b. Proposed System

MediQueueAI provides a frontend for accessing patient and clinical information. The frontend communicates with FastAPI REST endpoints, which interact with PostgreSQL to retrieve and update records. When an assessment is requested, the clinical inputs are processed by the assessment component and translated into risk and priority outputs. The result can then be used to prioritize the queue and support treatment workflow updates, as shown in Fig. 1.

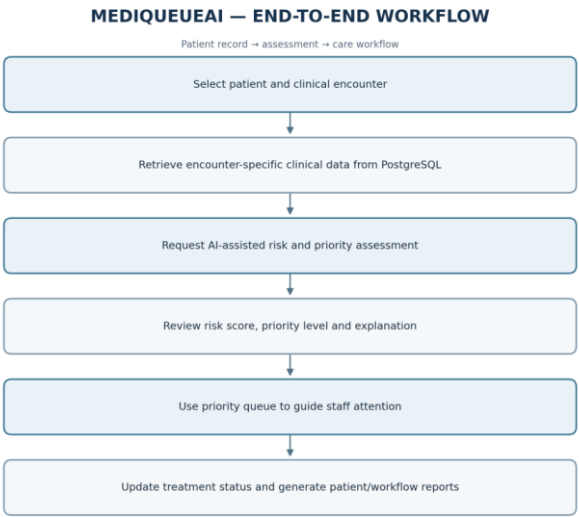


Fig. 1. End-to-end MediQueueAI workflow.

1.c. Functional Requirements

Table II lists the functional requirements identified for the prototype.

TABLE II. FUNCTIONAL REQUIREMENTS

ID	Requirement	Description
FR-01	Patient management	Search patient records and add new patient information.
FR-02	Encounter mapping	Retrieve clinical information for the correct patient encounter.
FR-03	Clinical inputs	Use symptoms and vital signs including heart rate, blood pressure, SpO ₂ , respiratory rate, and temperature.
FR-04	AI assessment	Produce risk score, risk level, priority score, priority level, and an explanation when requested.
FR-05	Priority queue	Order cases according to priority and reflect queue changes.
FR-06	Doctor assignment	Support doctor assignment as part of the care workflow.
FR-07	Treatment tracking	Record or display treatment status and completion.
FR-08	Reports	Connect patient and encounter information with patient reports and workflow reporting.
FR-09	System status	Show API health and PostgreSQL connectivity.

1.d. Non-Functional Requirements

Table III summarizes the design considerations that apply beyond specific features.

TABLE III. NON-FUNCTIONAL REQUIREMENTS

Requirement	Design consideration
Reliability	Ensure the displayed assessment belongs to the selected patient and encounter.
Usability	Present risk and priority categories in a clear, staff-oriented interface.
Maintainability	Keep frontend, API, database, and assessment responsibilities modular.
Scalability	Allow additional patients, doctors, departments, and encounters as the system grows.
Security and privacy	Protect sensitive health information; exact controls must be verified in the implementation.
Clinical safety	Display outputs as decision support and retain human clinical oversight.

1.e. Four-Stage Methodology

The methodology follows four main stages: clinical data, AI assessment, priority classification, and database update, as shown in Fig. 2. This sequence connects input information with a visible result and a subsequent workflow action .

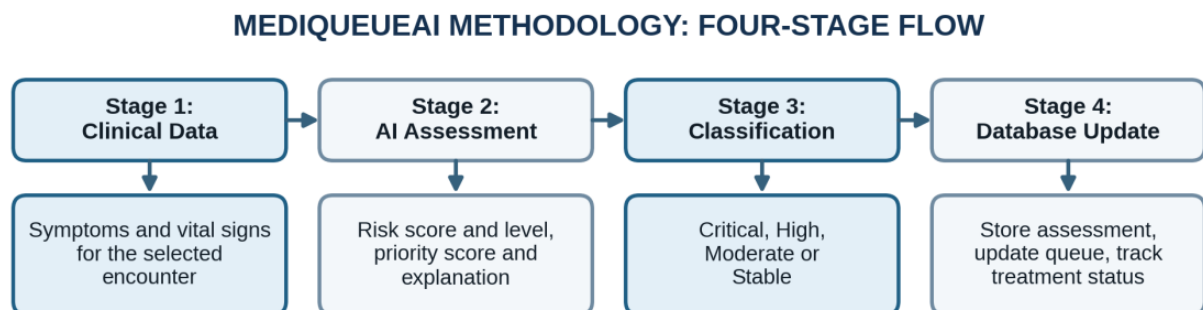


Fig. 2. Four-stage methodology flowchart.

1.f. Stage 1: Clinical Data

The assessment uses patient symptoms and vital signs. The inputs are heart rate, blood pressure, SpO₂, respiratory rate, temperature, and symptoms, as listed in Table IV. Correct patient-to-encounter mapping is emphasized so that the system does not associate the assessment with the wrong clinical record.

TABLE IV. CLINICAL INPUT DATA

Input	Purpose in the workflow
Heart rate	Provides a measured vital sign for the assessment.
Blood pressure	Provides a cardiovascular observation.
SpO ₂	Provides an oxygen saturation observation.
Respiratory rate	Provides a breathing-rate observation.
Temperature	Provides a body-temperature observation.
Symptoms	Adds reported clinical context to measured vital signs.

1.g. Data Retrieval and Verification

Patient and clinical data are retrieved from PostgreSQL, and the correct Patient → Encounter mapping is part of the verification path (Fig. 3). The assessment is performed only when requested. This is an important workflow distinction: stored clinical information is not itself an AI result; the result is generated from the selected encounter when the assessment action is invoked.

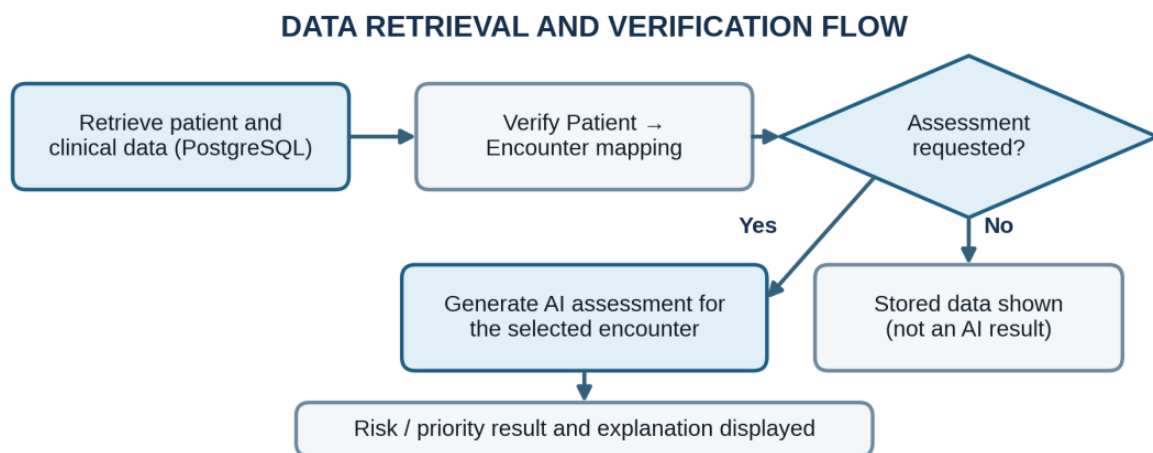


Fig. 3. Data retrieval and verification flowchart.

The database schema, exact SQL queries, missing-value policy, and validation rules are implementation details that are outside the scope of this report.

1.h. Stage 2: AI Assessment

After clinical data is retrieved, the assessment component generates a risk score and risk level, together with a priority score and priority level, and an explanation accompanies the assessment. This report describes the assessment component by its inputs and outputs; it does not attribute a specific model architecture, training procedure, threshold calibration, or clinical validation dataset to the implementation.

1.i. Stage 3: Priority Classification

Each assessed case is classified into one of four priority categories, defined in Table V. Fig. 4 shows the path from clinical inputs to the final classification.

TABLE V. RISK AND PRIORITY LEVELS

Priority category	Meaning
Critical	Immediate attention required.
High	Urgent attention.
Moderate	Requires monitoring.
Stable	Routine care.

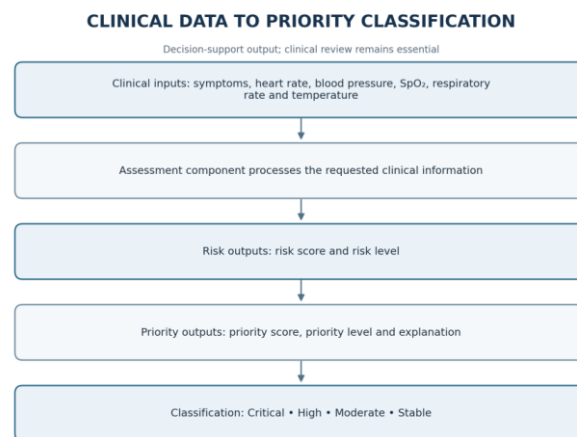


Fig. 4. Clinical data to risk and priority classification flowchart.

Risk and priority are presented as related but distinct outputs. The risk score and risk level summarize the assessment, while the priority score and priority level identify the pathway for organizing care. A high or critical output should prompt appropriate review by qualified healthcare personnel; it should not independently determine diagnosis or treatment. Any real-world deployment would require validation of input ranges, threshold selection, explanation quality, performance across patient groups, and safe handling of missing or inconsistent data.

1.j. Stage 4: Database Update and Priority Queue

In the final stage the assessment is stored, the queue is updated, and treatment status is tracked in PostgreSQL. This step closes the loop between the generated result and the operational workflow. Completed cases are removed from the active priority queue, as shown in Fig. 5.

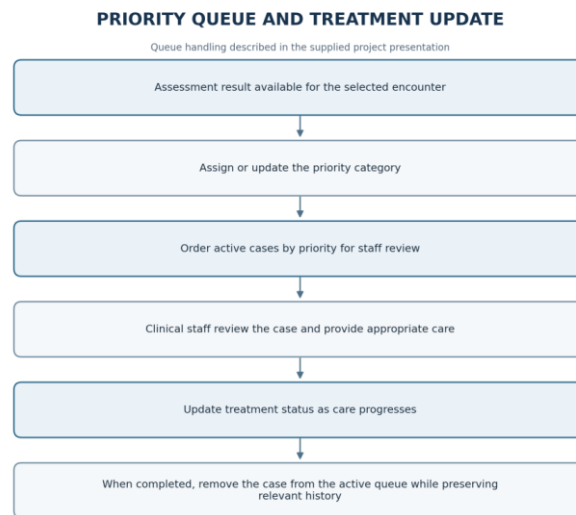


Fig. 5. Priority queue update flowchart.

A robust queue workflow must preserve the link between patient, encounter, assessment, and treatment status. Updating a queue entry should not overwrite a different encounter or remove the wrong patient. A completed case is removed from the active queue, while its historical encounter and assessment information should remain available where the application supports record history.

1.k. System Architecture

The system uses a layered architecture linking the user interface, API services, PostgreSQL database, assessment component, and care workflow (Fig. 6). The frontend is built with React, TypeScript, and Vite; the backend exposes FastAPI REST APIs; and PostgreSQL stores patient and clinical information [2].

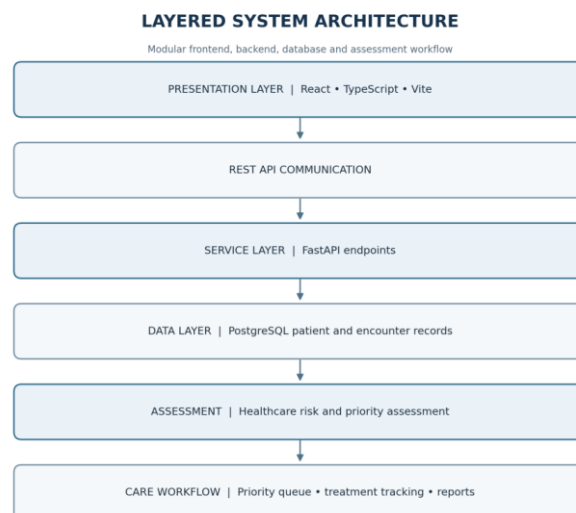


Fig. 6. Layered system architecture.

Separating the frontend, API, database, and assessment responsibilities makes the system easier to reason about and maintain. The frontend focuses on presenting records and actions; the API mediates requests; PostgreSQL persists structured information; and the assessment component produces risk and priority outputs. This modular design also provides a basis for future expansion.

The design emphasizes actual database-connected information rather than static demonstration data, and the correct patient-to-encounter relationship is central to this. In a healthcare context, the system should validate identifiers and ensure that assessment, prescriptions, treatment status, and reports reference the intended encounter.

The technologies used in each layer are listed in Table VI. FastAPI [3] provides the backend REST services, PostgreSQL [4] is the persistent database, and the frontend is developed with React [5], TypeScript [6], and Vite [7].

TABLE VI. TECHNOLOGY STACK

Layer / role	Technology	Use in the project
Frontend	React	User interface for patient and workflow management.
Frontend language	TypeScript	Typed frontend development.
Frontend tooling	Vite	Frontend development and build tooling.
Backend	FastAPI	Backend services and REST APIs.
Database	PostgreSQL	Persistent patient, encounter, clinical, and workflow data.
Communication	REST APIs	Connect frontend requests with backend services.
AI component	Healthcare Risk & Priority Assessment	Generate risk and priority outputs from symptoms and vitals.
Development environment	VS Code / Antigravity	Tools used during development.

The frontend communicates with the backend through REST APIs. The interactions implied by the workflow include retrieving patient or encounter details, submitting an assessment request, receiving risk and priority results, and updating treatment status. Endpoint paths, request fields, response schemas, and error codes are documented in the project repository [1].

1.m. Data Model and Information Flow

Table VII describes the logical entities of the system. It is a conceptual description of the information model; the table names and foreign keys of the implemented schema are maintained in the repository [1].

TABLE VII. CORE DATA ENTITIES AND RELATIONSHIPS

Logical entity	Information associated with it	Relationship / note
Patient	Patient identifier and demographic/registration information.	One patient may have multiple encounters.
Encounter	Visit or clinical encounter identifier and context.	Connects the selected patient to encounter-level data.
Clinical data	Symptoms and vital signs such as HR, BP, SpO ₂ , RR, and temperature.	Input to an assessment request.
Assessment	Risk score/level, priority score/level, and explanation.	References the encounter assessed.
Prescription	Prescription information associated with care.	Remains connected to the selected encounter.
Treatment status	In-progress or completed workflow status.	Used to track care and update the active queue.
Report	Patient/workflow report output.	Remains traceable to relevant patient/encounter records.

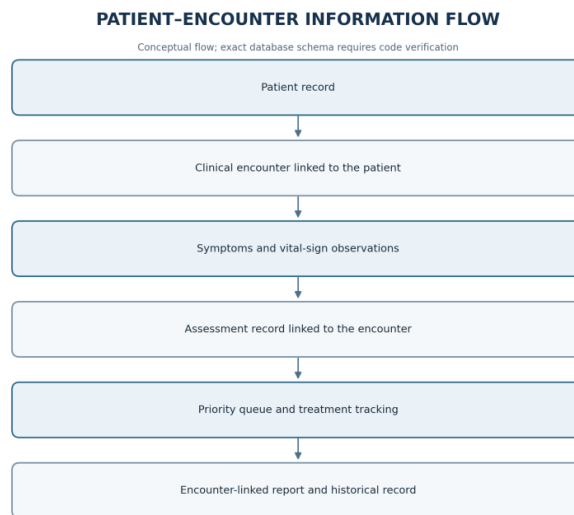


Fig. 7. Patient–encounter information flow.

The conceptual model (Fig. 7) emphasizes encounter-level traceability. A patient may have more than one encounter over time, so assessment outputs should not be stored or displayed as though they apply to every encounter. Prescriptions, treatment status, and reports are associated with the relevant selected encounter. The following database constraints are important for this design:

- Unique identifiers for patients, encounters, and assessment records.
- Foreign-key relationships between encounters and patients, and between assessments and encounters.
- Validation of clinical units and acceptable input formats.
- Consistent handling of missing, stale, duplicated, or contradictory observations.
- Appropriate access control, backups, auditability, and protection of health data.

2. EXPERIMENTS

2.a. Core Modules

The prototype is organized into several modules that together form the intended healthcare workflow (Table VIII). Fig. 8 shows an illustrative dashboard interface built with sample data.



Fig. 8. Illustrative MediQueueAI dashboard interface (sample data).

TABLE VIII. CORE MODULES OF THE PROTOTYPE

Module	Function
Patient management	Search records, view history, and add new patients.
Encounter and clinical data	Retrieve clinical information for the correct patient encounter.
AI assessment	Display risk score, priority level, and an assessment explanation.
Priority queue	Identify critical/high-risk patients and show queue status.
Doctor assignment	Connect patients with an appropriate doctor as part of the workflow.
Treatment tracking	Update treatment progress and completion status.
Reports and analytics	Provide patient statistics, risk distribution, and treatment outcome information.
System status	Monitor API health and PostgreSQL database connectivity.

2.b. End-to-End Interaction

A typical session with the system follows these steps:

- Select or search for the patient and the relevant encounter.
- Retrieve the encounter's clinical data from PostgreSQL.
- Request the risk and priority assessment when appropriate.
- Review the risk score, risk level, priority score, priority level, and explanation.
- Use the priority queue to identify the appropriate care pathway.
- Update treatment status as the case progresses.
- Generate or review reports linked to the patient and encounter.

2.c. Demonstration Case

A critical emergency case for the demonstration patient Rahul Verma is used to illustrate the workflow (Fig. 9). The values in Table IX reproduce the demonstration example; they are not a clinical recommendation and should not be reused as a real patient record. The resulting risk and priority summary is shown in Fig. 10.

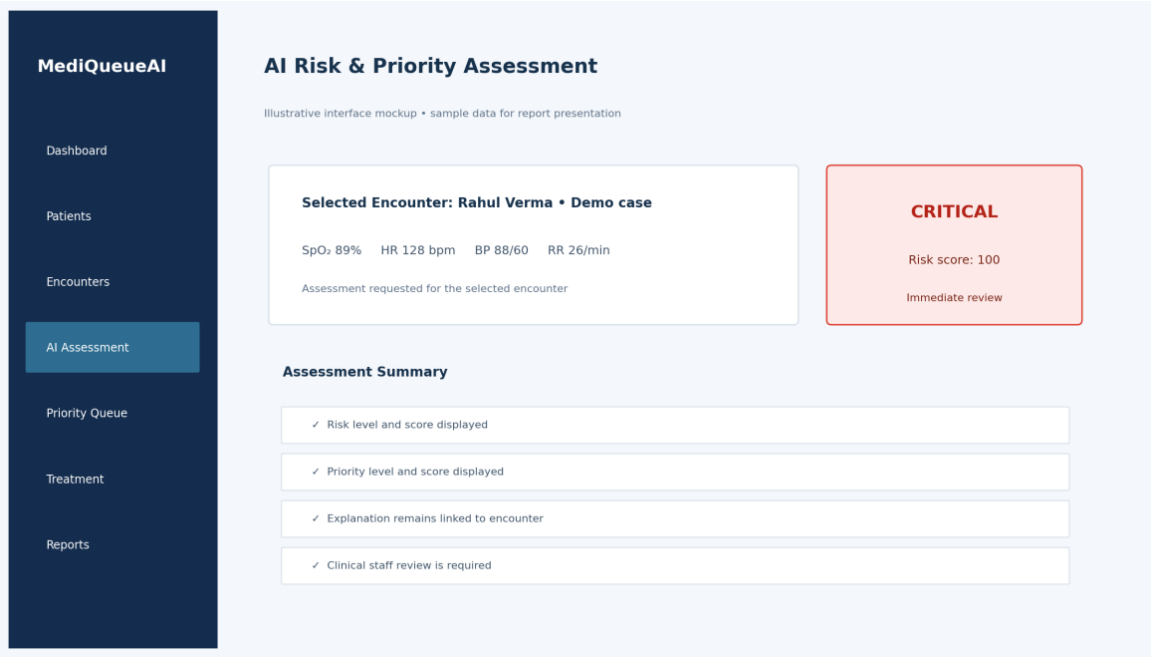


Fig. 9. Illustrative AI risk and priority assessment screen (sample data).

TABLE IX. DEMONSTRATION CASE VALUES

Parameter	Demonstration value
SpO ₂	89%
Heart rate	128 beats/min
Blood pressure	88/60
Respiratory rate	26/min
Risk score	100
Priority level	CRITICAL — Immediate Attention Required

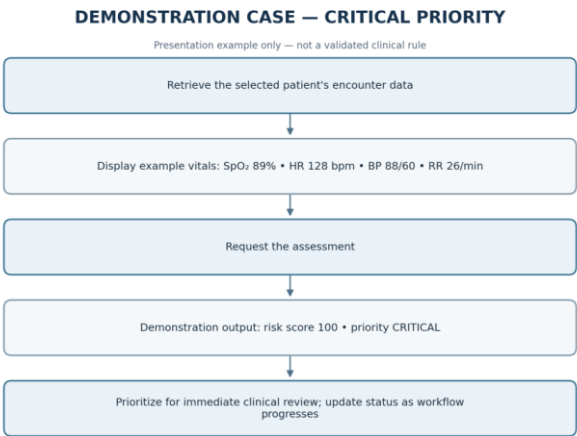


Fig. 10. Demonstration case risk–priority summary.

2.d. Demonstration Walkthrough

The demonstration proceeds through the following checks:

- Retrieve the patient's clinical data from PostgreSQL.
- Confirm that the displayed data belongs to the intended encounter.
- Trigger and display the assessment result.
- Verify that the critical priority is reflected in the queue ordering.
- Update treatment status and verify that completed cases leave the active queue.

2.e. Interpretation

The example demonstrates how the project connects clinical inputs, risk and priority outputs, and queue management. A second example emphasizes encounter-level data retrieval and the relationship between assessment, prescriptions, treatment status, and patient reports. The interface figures in this report are illustrative mockups drawn with sample data to communicate the intended screens. No formal test report, ground-truth labels, or measured accuracy metrics are reported for the prototype.

3. RESULTS

3.a. Observed Prototype Capabilities

The prototype demonstrates a connected workflow for patient management, clinical data retrieval, AI-assisted risk assessment, priority classification, queue ordering, treatment tracking, and reporting. Fig. 11 shows the emergency priority queue, in which active cases are ordered by priority. The strongest design feature is the verification path from patient data to the selected encounter and onward to assessment and the care workflow.

Priority	Patient	Encounter	Vitals summary	Queue status
CRITICAL	Rahul Verma	#14	SpO ₂ 89%, HR 128	Immediate review
HIGH	Patient P012	#18	Vitals require review	Urgent
MODERATE	Sreeja Valluru	#11	Encounter reviewed	Monitor
STABLE	Patient P007	#16	No urgent flag	Routine

Fig. 11. Illustrative emergency priority queue interface (sample data).

3.b. Benefits

The benefits expected from the approach are:

- Faster identification of cases marked as high risk or critical by the assessment.
- More visible prioritization of patients according to risk and priority levels.
- Reduced repetitive searching through centralized patient information.

- Improved organization of clinical and administrative workflow.
- Potential to extend the system to additional patients, doctors, departments, and facilities.

3.c. Evaluation Summary

Table X summarizes the evaluation dimensions, the evidence available for each, and how each should be interpreted.

TABLE X. EVALUATION SUMMARY

Evaluation dimension	Evidence available	Interpretation
Data traceability	Correct Patient → Encounter mapping is explicitly part of the design.	A key workflow requirement; verify with integration tests.
Assessment output	Risk score/level and priority score/level are produced.	Demonstrates the intended output; not evidence of clinical validity.
Priority handling	Critical, High, Moderate, and Stable categories are used.	Useful for workflow display; thresholds require validation.
Database integration	PostgreSQL and database-connected information are used.	Supports the design claim; schema and integrity tests are needed.
Workflow completion	Treatment-status update and queue removal are part of the demonstration flow.	Verify behavior through repeatable test cases.
Performance	No benchmark or formal metrics are reported.	No quantified speed, accuracy, or reliability is claimed.

3.d. Discussion

The project is best understood as a healthcare information management prototype with AI-assisted triage support. Its value depends not only on the assessment output but also on correct data mapping, clear presentation, reliable queue updates, and human review. Further evaluation should include functional testing, API error handling, database consistency checks, usability feedback, and clinical safety review.

4. CONCLUSION AND FUTURE WORK

MediQueueAI presents a database-driven approach to healthcare and patient information management that connects patient encounters, clinical observations, AI-assisted risk assessment, priority classification, queue handling, treatment status, and reporting. The system uses React, TypeScript, and Vite for the frontend, FastAPI for REST services, and PostgreSQL for persistent data.

The methodology is organized around clinical data retrieval, assessment, classification into Critical, High, Moderate, and Stable categories, and database updates that affect the priority queue and treatment workflow. The demonstration case illustrates how a critical label moves an urgent case to the front of the care workflow, and the design emphasizes traceability from the correct patient encounter to the assessment and subsequent records.

The project can help organize information and make urgency more visible to staff, but the evidence presented here does not establish clinical accuracy or production readiness. Before real-world use, the system would need verified implementation documentation, systematic software testing, data-protection controls, performance evaluation, and clinical validation. Human oversight must remain central to decisions about diagnosis and treatment.

4.a. Limitations, Clinical Safety and Privacy

This report describes the architecture, workflow, technology stack, features, and a demonstration case. It does not establish the assessment algorithm, model training data, model evaluation metrics, clinical validation, exact database schema, security implementation, or production deployment status.

Healthcare data is sensitive. A real deployment would require appropriate authentication and authorization, least-privilege access, secure transport, protected storage, audit logging, backup and recovery procedures, and a policy for retention and deletion. The assessment must be treated as decision support; qualified clinicians remain responsible for interpreting observations and deciding care. The category labels alone do not prove that the system can safely triage real patients.

4.b. Future Work

Table XI lists recommended enhancements. They are recommendations for further development, not claims that these capabilities are already implemented.

TABLE XI. LIMITATIONS AND FUTURE ENHANCEMENTS

Area	Possible enhancement
Assessment validation	Document the algorithm, thresholds, intended use, limitations, and clinical review process.
Testing	Add unit, API, database integration, edge-case, and regression tests.
Data quality	Validate units, ranges, missing values, conflicting observations, and encounter identity.
Security	Review authentication, authorization, encryption, audit trails, and secrets management.
Explainability	Provide clear reasons for each priority output and disclose uncertainty where available.
Analytics	Add validated summaries for queue trends, risk distribution, and treatment outcomes.
Scalability	Evaluate performance as patient, encounter, department, and facility counts grow.
Usability	Conduct feedback sessions with representative healthcare staff.

REFERENCES

- [1] “Healthcare Management System / MediQueueAI,” GitHub repository. [Online]. Available: https://github.com/sreejavalluru96-max/Healthcare_management_system.
- [2] “MediQueueAI: Healthcare Management System,” GitHub Pages. [Online]. Available: https://sreejavalluru96-max.github.io/Healthcare_management_system/.
- [3] FastAPI, “FastAPI Documentation.” [Online]. Available: <https://fastapi.tiangolo.com/>.
- [4] PostgreSQL Global Development Group, “PostgreSQL Documentation.” [Online]. Available: <https://www.postgresql.org/docs/>.
- [5] React, “React Documentation.” [Online]. Available: <https://react.dev/>.
- [6] TypeScript, “TypeScript Documentation.” [Online]. Available: <https://www.typescriptlang.org/docs/>.
- [7] Vite, “Vite Guide.” [Online]. Available: <https://vite.dev/guide/>.